Construyendo Agentes con Google ADK

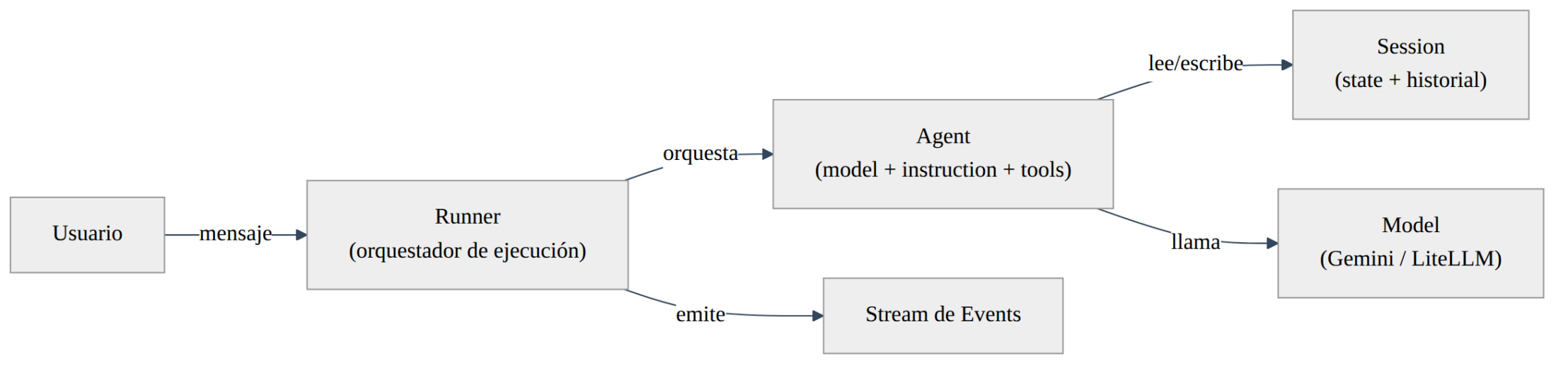
De un agente con una tool a un pipeline de 5 agentes en paralelo

Curso de Arquitectura de Agentes — UTDT / UNLP

Seis módulos, cuatro abstracciones, ningún concepto que no se repita.

Las 4 piezas de ADK

Todo lo de hoy — desde el ejemplo más simple hasta el pipeline final — es una combinación de las mismas cuatro abstracciones.



- **Agent** — modelo + instrucción + tools opcionales. **No tiene estado propio ni persistencia**: es una unidad de transformación que, dada una conversación hasta cierto punto, produce su continuación (texto, pedido de función, o estructura validada).
- **Runner** — cubre exactamente lo que el agente no cubre: recibe el mensaje, ejecuta las tools que el modelo pide, reinserta el resultado en la conversación y emite `Event`s a medida que avanza.
- **Session** — historial de eventos + un diccionario de estado (`session.state`) sobre el que todos los agentes de una misma conversación leen y escriben.
- **Model** — la interfaz al proveedor generativo. Gemini nativo por default, sustituible de forma transparente.

Agentes LLM · *Runtime / Runner* · *Sessions* · *State*

Qué es `adk web` realmente

`adk web` = Runner + InMemorySessionService + UI de chat

- Es un servidor local que instancia un `Runner` con un `InMemorySessionService` **por ustedes**, y le pone una UI de chat encima.
- El selector de arriba a la izquierda elige qué `root_agent` recibe ese `Runner` — nada más que eso.
- El **panel de eventos** de la derecha es literalmente el stream de `Event`s que emite el `Runner`, renderizado.
- Nada de lo que van a ver hoy es una simplificación didáctica: es la traza de ejecución real.

> *El diseño basado en flujo de eventos, y no en una llamada que devuelve un único valor, es lo que permite que la misma maquinaria sirva para una UI que muestra cada paso intermedio y para un script que solo espera la respuesta final.*

1. `tools_agent`

Tool calling básico

Pregunta que resuelve: ¿cómo le doy a un LLM la capacidad de *hacer* algo — consultar una API, leer un archivo, hacer una cuenta — en vez de solo generar texto?

- Cualquier función Python común puede ser una tool: **sin decoradores, sin registros explícitos, sin wrappers**.
- ADK inspecciona la función por reflexión — nombre, type hints de los parámetros, valores por defecto y docstring — y construye automáticamente un JSON Schema que la describe ante el modelo.
- Ese schema viaja en cada turno como parte del contexto, de modo que el modelo decide **por sí mismo** si conviene invocar alguna herramienta y con qué argumentos.

El código: el docstring es el prompt

```python
def get_current_date_and_time() -> dict:
    """Returns the current date and time in a dictionary format."""
    ...

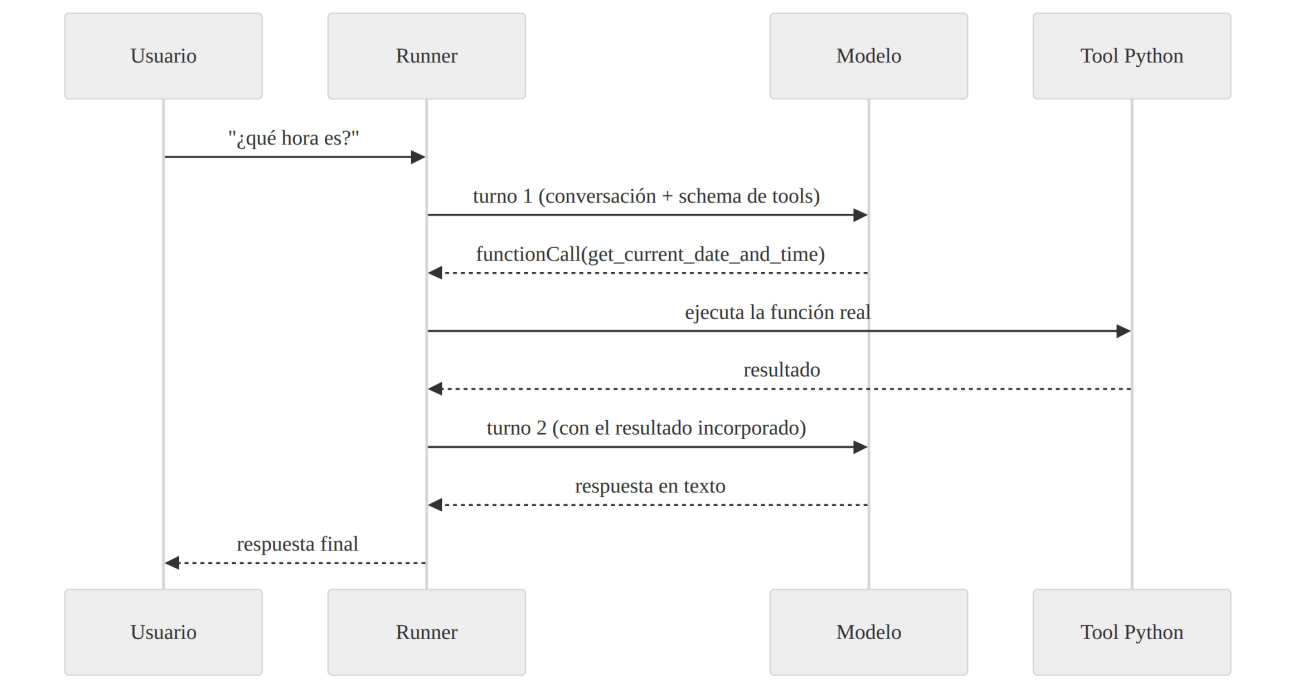
root_agent = Agent(
    name="tools_agent",
    tools=[get_current_date_and_time, get_randomuser_from_ramdomuserme],
    model="gemini-2.5-flash",
)
```

> *El **docstring** no es documentación para quien lee el código — es, en sentido estricto, parte del texto que el modelo procesa para decidir.*

Si esa documentación es vaga o ambigua, el modelo se queda sin información para decidir cuándo invocar la función: lo más probable es que responda especulando, o que llame a la herramienta equivocada. **La calidad del docstring de una tool equivale en importancia a la calidad de una instrucción de sistema.**

Function tools — "The docstring of your function serves as the tool's description and is sent to the LLM."

El ciclo de tool calling: dos turnos con el modelo



- En el **turno 1** el modelo no produce texto: produce una solicitud de invocación con la tool elegida y sus argumentos.
- El `Runner` intercepta esa solicitud, ejecuta la función Python real y arma un turno nuevo informando el resultado.
- Recién en el **turno 2** el modelo genera lenguaje natural, ya informado por el resultado real.
- Todo esto ocurre **sin una sola línea de enrutamiento condicional** escrita por el desarrollador: la decisión es enteramente del modelo.

Demo en vivo: `adk web` → `tools_agent` → *"¿qué hora es?"*, *"dame un usuario random"*, y si da el tiempo *"decime la hora y de paso un usuario de prueba"* (dispara ambas tools en un mismo turno, en el orden que el modelo elija).

2. `structured_output`

Salida estructurada + 2 agentes encadenados

Problema: hasta acá la respuesta del agente es siempre texto libre. Excelente para un chat, frágil cuando esa salida tiene que ser el *input* de otro programa, de otra función o de otro agente — significa parsear con regex y rezar.

Solución: forzar la salida del modelo a un schema de datos tipado, apoyándose en **Pydantic**.

Este módulo introduce además la segunda gran idea del curso: **cómo se encadenan dos agentes** que, en rigor, son unidades de ejecución completamente independientes y solo se comunican a través del estado compartido de la sesión.

Qué es Pydantic en este contexto

Modelos de datos tipados, validados en runtime

```python
class ConsultantTypeEnum(str, PyEnum):
    PSYCHOLOGIST = "psychologist"
    GENERAL_HELPER = "general_helper"

class ProblemAnalysis(BaseModel):
    consultant_type: ConsultantTypeEnum
    identified_issues_summary: str
```

`BaseModel` de Pydantic le da a ADK tres cosas:

1. **JSON Schema** generado automáticamente a partir de la clase → ADK lo incorpora al contexto del modelo como contrato de formato.
2. **Validación** del texto devuelto: se parsea como JSON y se valida contra la clase. Si falta un campo obligatorio, si un tipo no matchea, o si aparece un valor fuera de un Enum → **la ejecución falla explícitamente**.
3. Es un **contrato en sentido fuerte**, no una sugerencia de formato que el modelo pueda o no respetar.

El `Enum` merece atención aparte: al declarar un campo con un tipo enumerado se restringe *categóricamente* el espacio de respuesta del modelo para ese campo. No puede inventar una categoría — evita la proliferación de variantes textuales equivalentes que romperían cualquier procesamiento posterior.

> `output_schema` + `tools` en el mismo agente: confiable solo en Gemini 3.0+ (por eso ningún agente de este ejemplo mezcla ambos).

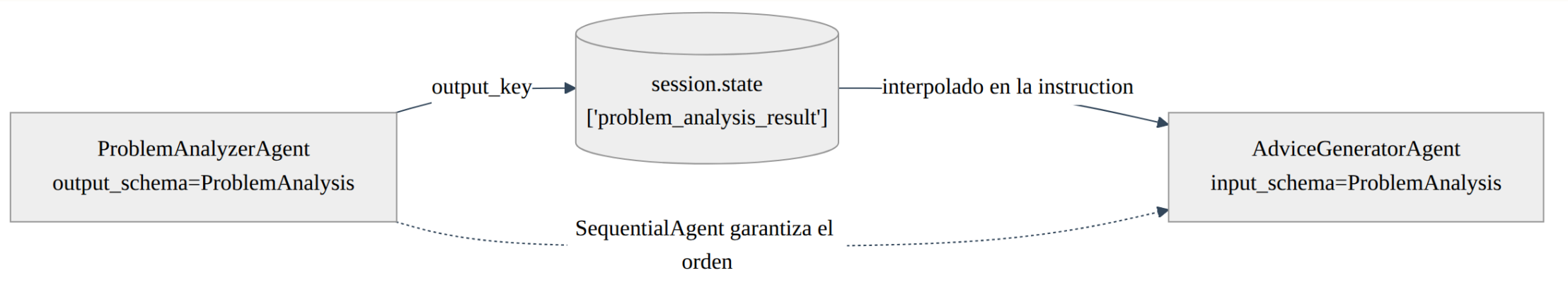
LLM Agents — `output_schema` / `input_schema` / `output_key`

El pase de posta tipado entre dos agentes

```
problem_analyzer_agent = LlmAgent(
    output_schema=ProblemAnalysis,
    output_key="problem_analysis_result",   # -> session.state
)

advice_generator_agent = LlmAgent(
    instruction="... {problem_analysis_result} ...",  # <- interpolado
    input_schema=ProblemAnalysis,
    output_schema=ConsultationResp,
)

root_agent = SequentialAgent(
    sub_agents=[problem_analyzer_agent, advice_generator_agent],
)
```

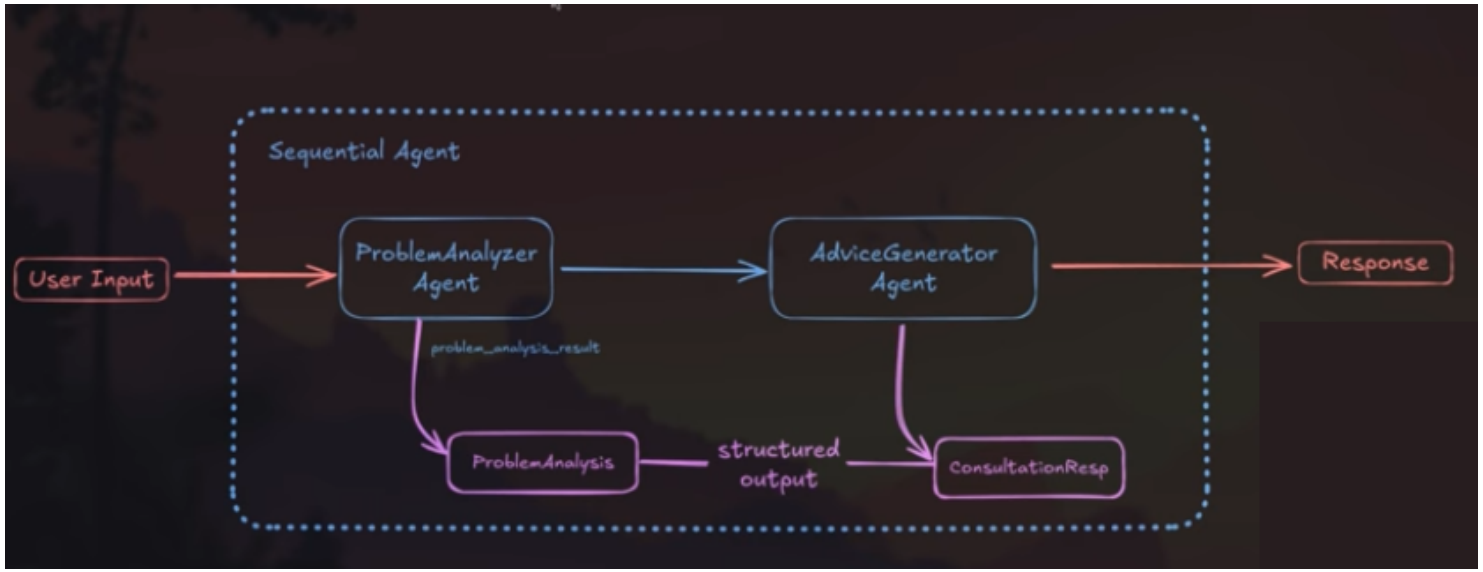


- `output_key` guarda automáticamente la salida en `session.state` bajo esa clave.
- El segundo agente referencia `{problem_analysis_result}` en el texto de su instrucción: ADK lo interpola desde el estado antes de armar el prompt, como un f-string resuelto por el framework.
- `input_schema` cierra el circuito: valida que lo inyectado tenga la forma esperada. Tipado extremo a extremo.
- `SequentialAgent` no razona: ejecuta sus `sub_agents` en orden estricto y garantiza que cada uno **termine de escribir en el estado** antes de que arranque el siguiente.

Demo: `adk web` → `structured_output`

> *Consulta: "no sé cómo organizar mis finanzas"*

- Abrir el evento del primer agente: **JSON crudo** de `ProblemAnalysis`, con el Enum resuelto a un valor válido de la lista.
- Mostrar cómo el segundo agente arma `ConsultationResp` a partir de eso.
- Abrir el ícono del grafo (aparece después del primer mensaje): una cadena de 2 nodos.

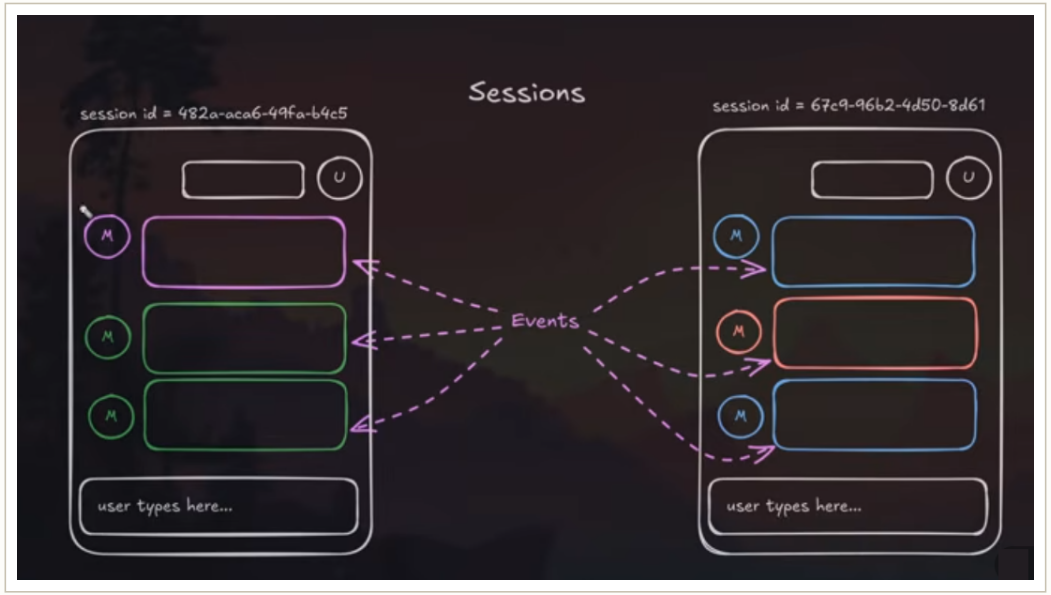


El grafo no es una ilustración aparte: es el mismo `SequentialAgent(sub_agents=[problem_analyzer_agent, advice_generator_agent])` de la slide anterior, dibujado nodo por nodo.

3. `sessions_and_agents`

El Runner y el SessionService, sin la UI de por medio

Este ejemplo se corre con `python run_agent_with_session.py` — **no** con `adk web`. La razón es pedagógica: ver exactamente lo que la UI esconde, instanciando el `SessionService`, creando la `Session` con estado inicial a mano y pasándole todo eso al `Runner` explícitamente.

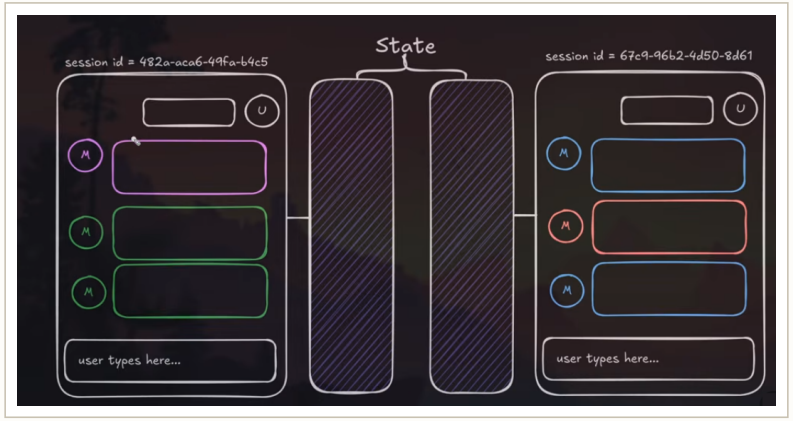


Dos sesiones (`482a-...` y `67c9-...`) son dos historiales de `Event`s **completamente aislados** entre sí: mismo agente, cero información compartida. Todo lo que sigue es *cómo* se construye y se mantiene ese aislamiento.

Instrucción parametrizada por estado

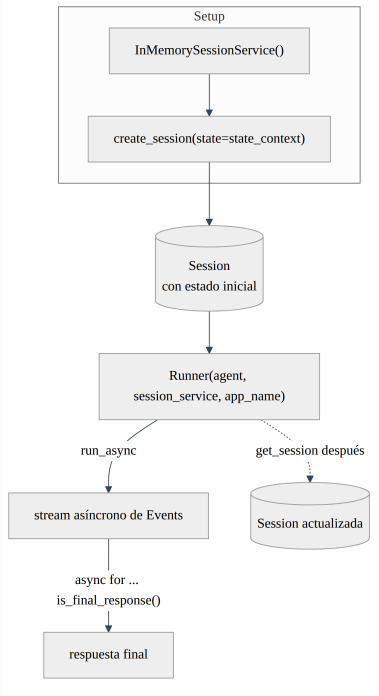
```
root_agent = Agent(
    name="PostAgent",
    instruction="""
    Name: {user_name}
    Post Preferences: {user_post_preferences}
    """,
)
```

Mismo mecanismo de interpolación que en `structured_output`. La novedad de este bloque no está en el agente, sino en **cómo se crea** ese estado.



El `State` de cada sesión **cuelga de esa sesión**, no es global. `{user_name}` y `{user_post_preferences}` se resuelven contra el estado de la sesión `482a-...`, nunca contra el de `67c9-...`, aunque ambas corran el mismo `PostAgent` al mismo tiempo.

El Runner explícito, paso a paso



Los 3 pasos del código:

1. `InMemorySessionService()` + `create_session(state=...)` — el estado inicial se carga **a mano**, antes de la primera interacción.
2. `Runner(agent=..., session_service=..., app_name=...)` + `run_async(...)` → un **generador asíncrono**: no devuelve una respuesta, devuelve un stream de `Event`s que hay que recorrer hasta encontrar `is_final_response()`.
3. `get_session(...)` al final → recuperar el `session.state` ya actualizado, exactamente como lo haría un backend que atiende un chat entre un request HTTP y el siguiente.

Runtime / Runner · *Sessions*

SessionService: tres sabores de persistencia

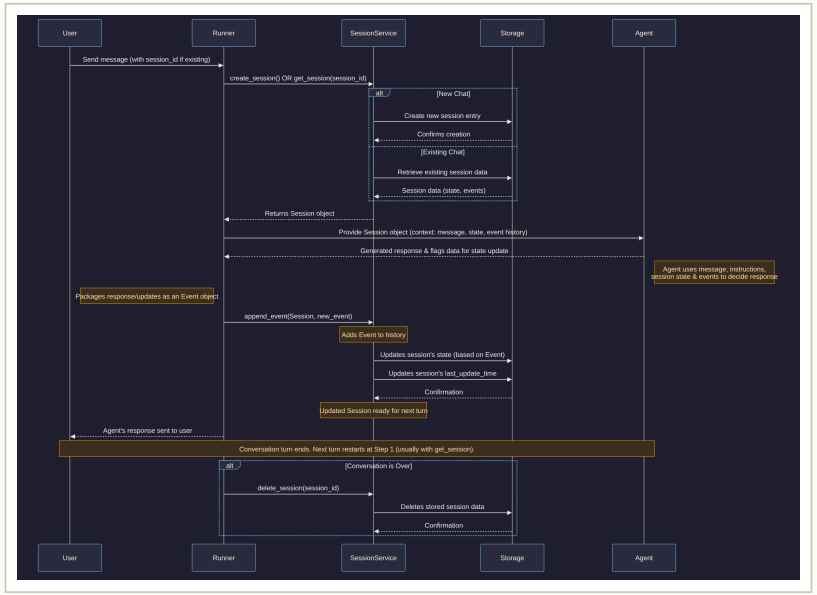
El `SessionService` es el responsable del **ciclo de vida** de las sesiones: crear (opcionalmente con estado inicial), recuperar por `app_name` + `user_id` + `session_id`, actualizar y eliminar.

Servicio	Persistencia	Cuándo
`InMemorySessionService`	Ninguna — vive y muere con el proceso	Desarrollo y demos
`DatabaseSessionService`	Base relacional	Producción con durabilidad
`VertexAiSessionService`	Infraestructura de Google Cloud	Producción con concurrencia

El punto conceptual del módulo: el estado de sesión puede poblarse **de manera completamente independiente de cualquier ejecución de agente**, con solo pasar un diccionario al crear la sesión. No es un subproducto exclusivo de los `output_key` del módulo anterior — es un espacio de datos de propósito general, poblable antes, durante o después de la ejecución, y recuperable en cualquier momento posterior consultando de nuevo al servicio.

Sessions · *State*

La traza completa de un turno



Cinco actores (`User`, `Runner`, `SessionService`, `Storage`, `Agent`) y cinco fases que corren en **cada turno de cualquier conversación**, tanto en el script explícito como en `adk web`:

1. **Resolver la sesión** — `create_session()` si es nueva, `get_session()` si existe (rama `alt` del diagrama). El `Storage` devuelve `state` + `events`.
2. **Generar la respuesta** — el `Runner` le pasa al `Agent` la `Session` completa: mensaje nuevo + estado + historial. Por eso `{user_name}` se puede interpolar: ya está ahí.
3. **Persistir el turno** — el `Runner` empaqueta todo como un `Event` y llama `append_event()`; el `Storage` agrega el evento al historial **y actualiza el estado en base a él** (acá es donde un `output_key` termina escrito).
4. **Responder y cerrar** — *"Next turn restarts at Step 1"*: **no hay conexión persistente entre turnos**. La memoria de la conversación no vive en el `Runner` ni en el `Agent` — vive enteramente en lo que el `SessionService` persistió.
5. **Cierre de conversación** (opcional, fuera del loop) — `delete_session()`.

4. `multi_model`

Orquestación multi-proveedor con LiteLLM

Problema: hasta ahora todos los agentes usaron Gemini nativo. En un sistema real conviene usar modelos distintos para tareas distintas.

Las tres motivaciones habituales:

- **Costo** — diferencias de precio por token entre proveedores para tareas de volumen.
- **Desempeño relativo** — un proveedor rinde mejor que otro en una tarea específica.
- **Contratos preexistentes** — el cliente ya tiene una relación comercial con determinado proveedor.

Este módulo introduce además la **segunda variante de orquestación multi-agente**: la ejecución en paralelo.

Qué es LiteLLM

Interfaz única sobre decenas de proveedores

```
LiteLlm(model="gemini/gemini-3.5-flash-lite")
LiteLlm(model="anthropic/claude-3-5-haiku-20241022")
LiteLlm(model="openai/gpt-4o-mini")
```

- El **prefijo antes de la barra** es una convención que LiteLLM interpreta para determinar tanto el endpoint de red como la variable de entorno de la que leer las credenciales.
- El formato de request es compatible con la API de OpenAI, sin importar cuál sea el backend real invocado por debajo.
- Para ADK, una instancia de `LiteLlm` es **intercambiable** con un string de modelo nativo: el agente y el resto del framework no distinguen un caso del otro. La diferencia queda encapsulada en esa clase.

> *Nota técnica: ADK emite un `UserWarning` al usar `LiteLlm` para enrutar a Gemini, sugiriendo el wrapper nativo. Es una recomendación de performance, no un error — acá se mantiene a propósito para preservar el patrón pedagógico sin necesitar API keys de terceros.*

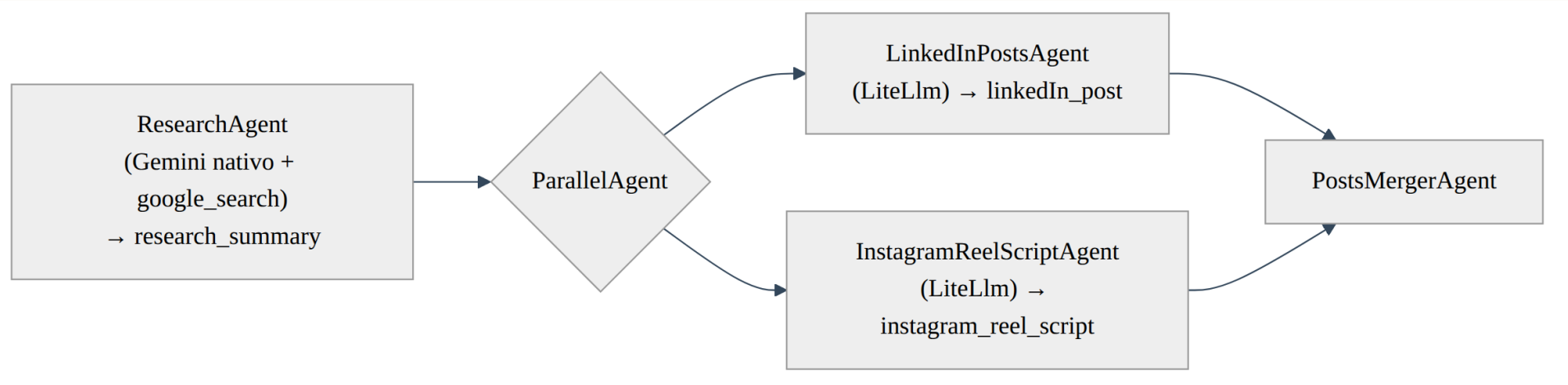
Modelos vía LiteLLM · *Modelos en general*

`Sequential` + `Parallel` anidados

```
researchAgent = Agent(model="gemini-2.5-flash", tools=[google_search], ...)  # nativo
linkedInAgent = Agent(model=LiteLlm(model="gemini/..."), ...)                # vía LiteLLM
instagramAgent = Agent(model=LiteLlm(model="gemini/..."), ...)               # vía LiteLLM

postsAgent = ParallelAgent(sub_agents=[linkedInAgent, instagramAgent])

root_agent = SequentialAgent(
    sub_agents=[researchAgent, postsAgent, postsMergerAgent],
)
```



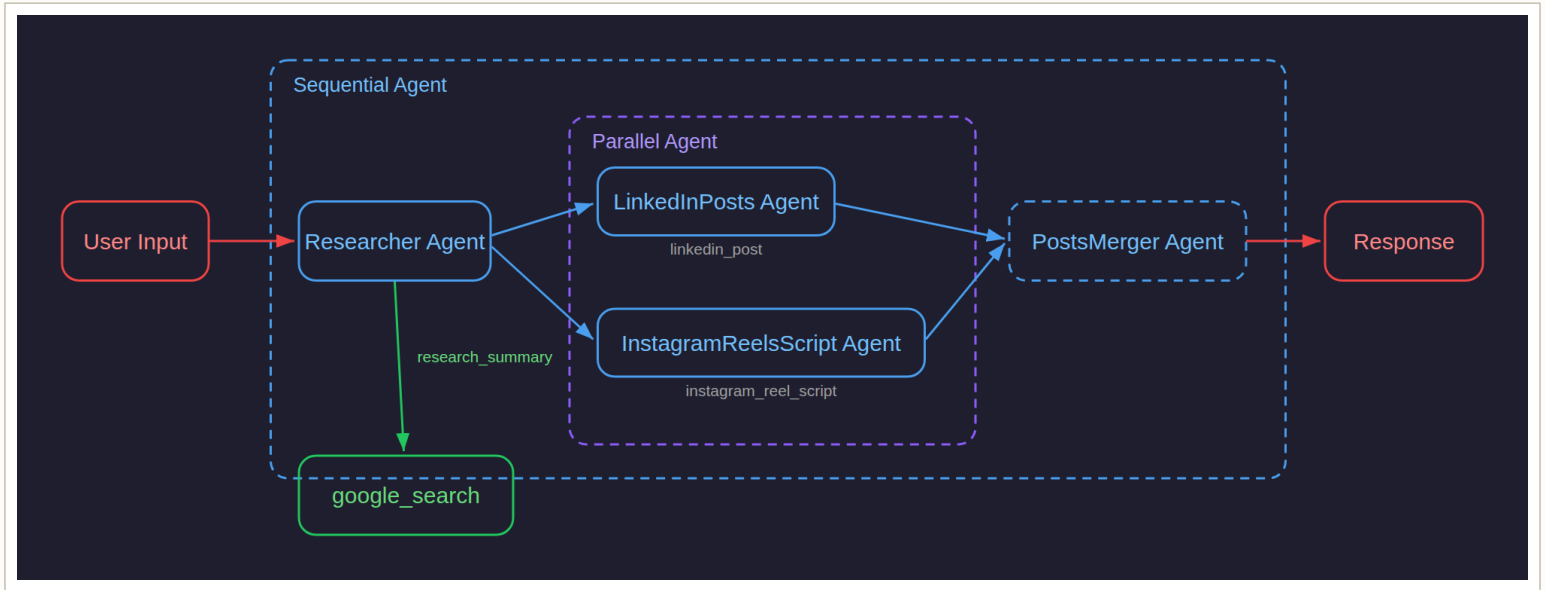
- `researchAgent` usa Gemini **nativo** porque necesita `google_search`, que solo funciona confiablemente con la integración nativa.
- `ParallelAgent` corre sus `sub_agents` **al mismo tiempo**: LinkedIn e Instagram no dependen uno del otro, ambos solo necesitan `research_summary`.
- Secuencial y paralelo **se anidan sin restricción**: la estructura de orquestación refleja la estructura real de dependencias de datos del problema.
- Los sub-agentes de un `ParallelAgent` deben escribir claves de estado **disjuntas** — si dos escriben la misma clave, gana el que termine último.

Parallel Agents

Demo: `adk web` → `multi_model`

> *Consulta: "el futuro de los agentes de IA"*

- `ResearchAgent` corre **solo** primero: los otros necesitan su `research_summary`.
- Luego **dos ramas simultáneas** — la primera vez en toda la clase que se ve paralelismo real en el panel de eventos.
- `PostsMergerAgent` fusiona ambas salidas al final.



La bifurcación/unión visual (recuadro punteado violeta) **es** el `ParallelAgent` anidado dentro del `SequentialAgent` externo: la misma estructura `research → (LinkedIn || Instagram) → merge` del código, dibujada.

Este ejemplo necesita el `.env` con Opción A (`GOOGLE_API_KEY` de AI Studio): LiteLLM en modo `gemini/` no usa credenciales de Vertex.

5. `agents_and_callbacks`

6 puntos de enganche en el ciclo de vida del agente

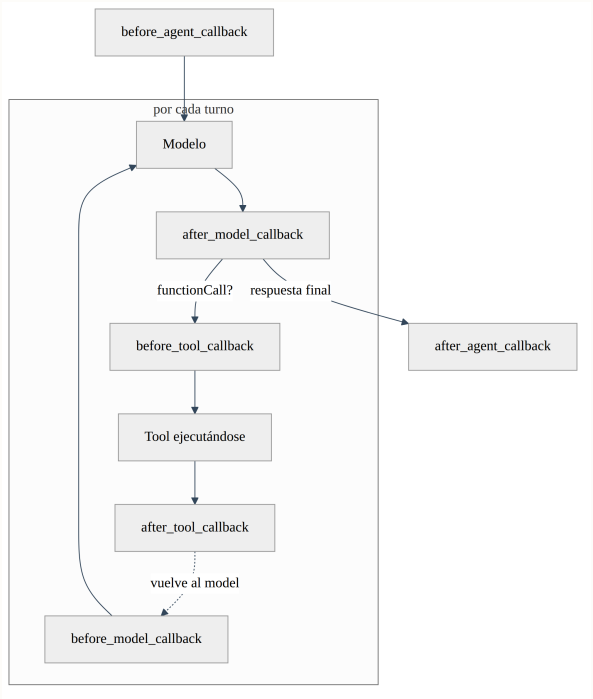
Hasta acá el agente fue tratado como una **caja negra**: se le entrega una conversación y en algún momento sale una respuesta, sin posibilidad de observar ni de intervenir en los pasos intermedios.

Los callbacks resuelven exactamente esa limitación. Son funciones que escribe el desarrollador y que ADK invoca automáticamente en seis momentos bien definidos, pasándoles el contexto relevante de esa etapa: el estado de la sesión, la request a punto de enviarse al modelo, la respuesta recién producida, o la tool y sus argumentos.

La regla, idéntica en los seis casos: `None` = *dejá pasar* (callback puramente observacional) · **objeto específico** = *override* del comportamiento por defecto.

Callbacks — "Returning a specific type of object (instead of signaling 'Continue') is how you override the ADK agent's default behavior."

El ciclo de vida completo, con los 6 callbacks



#	Callback	Momento	Ejemplo de uso
1	`before_agent_callback`	antes de procesar la solicitud completa	Inicializar contadores, timestamp
2	`after_agent_callback`	después de la respuesta final	Duración, métricas de lifecycle
3	`before_model_callback`	antes de cada llamada al LLM	Sanitización de PII
4	`after_model_callback`	después de cada respuesta del LLM	Enriquecimiento de la respuesta
5	`before_tool_callback`	antes de cada ejecución de tool	Validación de argumentos · caché (hit)
6	`after_tool_callback`	después de cada ejecución de tool	Formateo del resultado · caché (write)

Esta dualidad entre **observación pasiva** e **intervención activa** es el eje que organiza los cinco ejemplos que siguen.

Ejemplo 1: logging de ciclo de vida

```python
def before_agent_callback(callback_context) -> Optional[types.Content]:
    callback_context.state["session_id"] = ...
    callback_context.state["request_counter"] += 1
    print(f"[BEFORE AGENT] Interaction #{request_num} initiated.")
    return None   # <- puramente observacional

def after_agent_callback(callback_context) -> Optional[types.Content]:
    print(f"[AFTER AGENT] ... Duration: {duration_str}")
    return None
```

- El caso más simple: **ambos callbacks devuelven siempre `None`**, no interceptan nada, solo registran.
- El `CallbackContext` da acceso directo a `callback_context.state` — el mismo `session.state` de siempre.
- Se usa para inicializar un `session_id` la primera vez, llevar un contador de interacciones y un timestamp de inicio; el callback posterior lee ese timestamp y calcula la duración.
- Es la **base conceptual de cualquier sistema de logging o métricas** de un agente en producción, sin alterar en absoluto el comportamiento funcional.

Demo: mandar 2-3 mensajes cortos y señalar **en la terminal** (no en la UI) el `session_id` constante y el `request_counter` incrementando.

Ejemplo 2: sanitización de input (PII)

```python
def before_model_callback_sanitize(callback_context, llm_request) -> Optional[LlmResponse]:
    # busca PII (tarjeta, SSN) con regex en el último mensaje del usuario
    llm_request.contents[i].parts[0].text = sanitized_text   # muta in-place
    return None   # sigue el flujo, pero con el request YA modificado
```

- Aparece un segundo argumento: `llm_request`, el objeto que está a punto de mandarse al modelo, con toda la conversación en `llm_request.contents`.
- La detección es por regex simplificadas con fines demostrativos (tarjeta de crédito, SSN).
- **La sutileza clave:** el callback devuelve `None` — "seguí con el flujo normal" — pero ese flujo normal ya opera sobre una request distinta de la original, porque la modificación se hizo por **mutación directa del objeto** antes de retornar.
- **Gobernanza:** garantiza que cierta categoría de información nunca llegue efectivamente al proveedor externo del modelo, sin depender de que el modelo "decida" no usarla.

Demo: *"Mi tarjeta es 1234-5678-9012-3456, contame algo sobre ella"* → comparar en consola `Original user input` vs `Sanitized input to LLM` (`[REDACTED PII]`).

Ejemplo 3: enriquecimiento de la respuesta

```python
def after_model_callback_enhance(callback_context, llm_response) -> Optional[LlmResponse]:
    modified = copy.deepcopy(llm_response)
    # extrae datos estructurados con regex (ej: detalles de vuelo)
    # agrega links dinámicos según palabras clave detectadas
    modified.content.parts[0].text = current_text
    return modified   # <- reemplaza la respuesta del modelo
```

- Simétrico al ejemplo anterior, pero del lado de la **salida**: recibe la `llm_response` ya generada, antes de que llegue al usuario.
- Hace dos cosas: extrae con regex datos estructurados que el modelo devolvió en texto libre (y los guarda en `state` para uso posterior), y agrega links de ayuda según palabras clave.
- A diferencia del ejemplo 2, acá **sí retorna un objeto no nulo** — un `deepcopy` modificado — instruyendo explícitamente a ADK a sustituir la respuesta del modelo por la versión enriquecida.
- Patrón para post-procesar modelos **sin soporte nativo confiable de `output_schema`**, o para incorporar contenido dinámico que no tiene sentido delegar en la generación del modelo.

Demo: *"Book me a flight from London to Paris for 2025-12-25"* → log de `Extracted flight info` en consola y bloque "Flight Summary Logged" agregado al final de la respuesta en la UI.

Ejemplo 4: validación de argumentos de tool

```python
def before_tool_callback_schedule(tool, args, tool_context) -> Optional[Dict]:
    if formato_de_fecha_invalido:
        return {"result": "Error: formato inválido"}   # <- la tool NUNCA se ejecuta
    if "afternoon" in args["time"]:
        args["time"] = "14:00"   # <- normaliza, muta in-place
    return None   # <- ahora sí, ejecuta la tool con args ya limpios
```

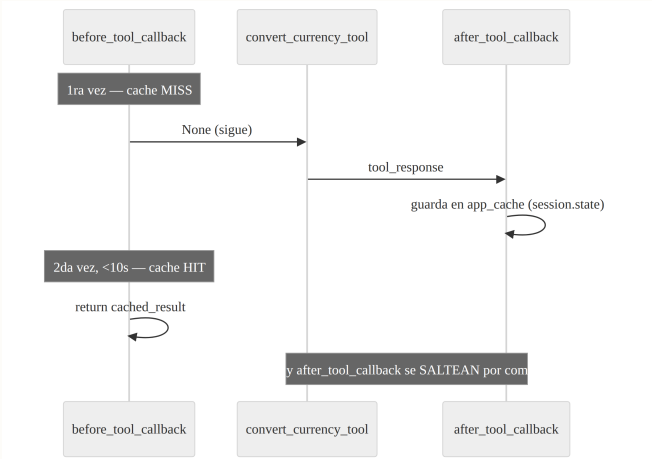
- Tercer par de callbacks: los de **tool**, no de agente ni de modelo. Recibe la `tool` que está por ejecutarse, los `args` que el modelo decidió pasarle (dict mutable) y el `ToolContext`.
- **Dos comportamientos según el retorno:** un dict de error hace que la tool real *jamás* se ejecute — el modelo recibe ese dict como si fuera el resultado genuino de la tool. Un `None` tras mutar `args` deja que la tool corra, pero con los argumentos ya normalizados.
- **Principio de diseño:** la validación debe ocurrir *antes* de invocar cualquier tool con efectos secundarios reales o costo asociado. Nunca descubrir un argumento inválido después de haber incurrido en ese costo.

Demo: dos turnos — uno con fecha inválida (`25/12/2025`) mostrando el bloqueo, y uno válido con *"afternoon"* mostrando la normalización a hora exacta antes de que la tool se ejecute.

Ejemplo 5: caching de resultados de tool

```python
async def before_tool_callback_cache(tool, args, tool_context) -> Optional[Dict]:
    if hay_cache_fresco(args):
        return {"cached_result": ...}   # <- SKIP la tool real (y también after_tool_callback)
    return None

async def after_tool_callback_format_and_cache(tool, args, tool_context, tool_response) -> Dict:
    tool_context.state["app_cache"][cache_key] = (timestamp, tool_response)
    return {"result_summary": "100.00 USD is approximately 92.00 EUR..."}
```



- La clave de caché es **determinística**: se deriva del nombre de la tool y sus argumentos.
- El "cache" **es** `session.state` — el mismo espacio usado en todos los módulos anteriores, empleado acá con otro propósito. **Cero infraestructura externa.**
- El punto pedagógico grande: `before_tool_callback` puede **eludir por completo** la ejecución real, sirviendo un resultado desde cualquier otra fuente. Es el mismo principio del ejemplo 4, aplicado no para bloquear una ejecución inválida sino para evitar una redundante.

Demo: 1) *"How much is 100 USD in EUR?"* → `Cache MISS` + `[TOOL EXECUTED]`. 2) Repetir dentro de 10s → `FRESH CACHED result`, el log `[TOOL EXECUTED]` **no aparece**. 3) Esperar >10s → `Cache EXPIRED`, la tool corre de nuevo.

6. `marketing_campaign_agent`

Cierre integrador: 5 agentes, cero conceptos nuevos

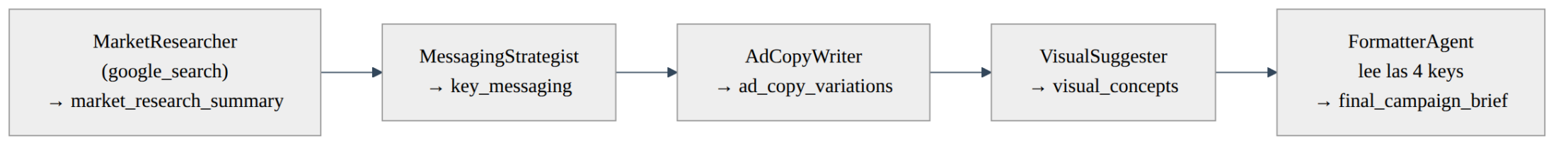
Este módulo **no introduce ningún concepto técnico adicional**, y su función pedagógica es precisamente demostrar esa ausencia de novedad.

Es la combinación de piezas ya vistas, escalada:

- `SequentialAgent` — del Módulo 2.
- `output_key` + interpolación de estado — de los Módulos 2 y 3.
- Una tool nativa (`google_search`) — del Módulo 1, ya usada en el Módulo 4.

Cinco agentes en cadena, de los cuales **solo el primero tiene una herramienta**. Los otros cuatro se limitan a razonar sobre la información acumulada en el estado por los que los precedieron.

Los 5 agentes en cadena




```
campaign_orchestrator = SequentialAgent(
    sub_agents=[
        market_research_agent, messaging_strategist_agent,
        ad_copy_writer_agent, visual_suggester_agent, formatter_agent,
    ]
)
```

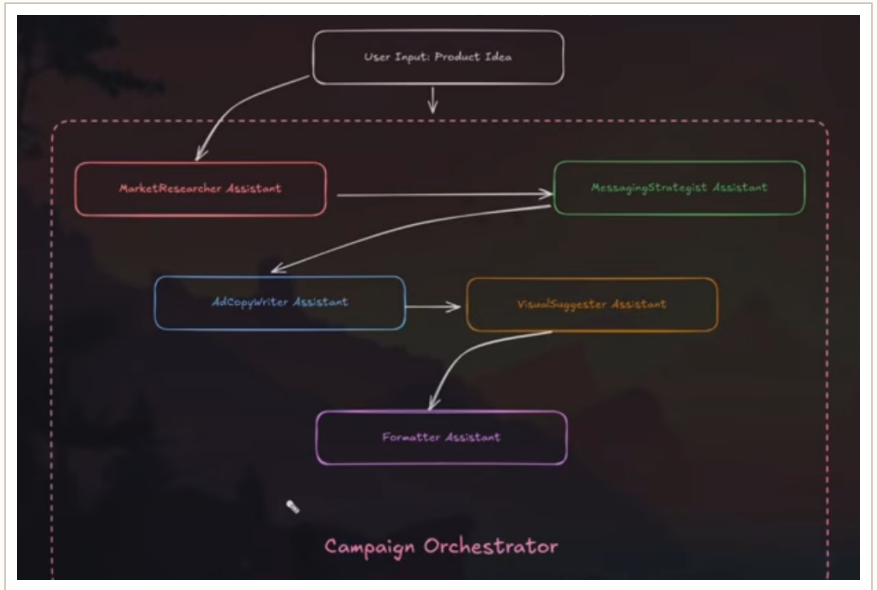
- Dos sintaxis para el **mismo mecanismo de fondo**: `structured_output` usaba `{clave}` interpolada; acá las instrucciones dicen en texto plano *"tu input está en* `state['market_research_summary']` *"* y ADK inyecta igual.
- El `FormatterAgent` es el único que necesita ver **las 4 claves a la vez**, no solo lo que dejó el inmediato anterior.
- Que un agente pueda leer múltiples claves producidas por agentes distintos, sin ninguna relación de comunicación directa entre ellos más allá de compartir la sesión, es exactamente lo que demuestra que **`session.state` no es un mecanismo de paso de parámetros entre funciones**: es un espacio de datos común, accesible por cualquier participante mientras la conversación siga activa.

Sequential Agents

Demo: `adk web` → `marketing_campaign_agent`

> *Consulta: "una app que ayuda a estudiantes a organizar sus apuntes con IA"*

- Panel de eventos: los 5 agentes completándose en **orden estricto** (tarda más — 5 llamadas al modelo, una con búsqueda real).
- Grafo: 5 nodos encadenados, resaltándose en verde a medida que corren.
- Brief final en Markdown: 4 secciones = 4 `output_key` distintos, concatenados por el `formatter_agent`.



Comparar mentalmente con el grafo de 2 nodos de `structured_output` y con el grafo bifurcado de `multi_model`: **misma forma, sin ningún `ParallelAgent` en el medio**.

Nota: si el grafo no aparece, verificar `graphviz` instalado (`dot -V`) y el paquete Python `graphviz` en el venv.

Cierre: las piezas, recapituladas

Pieza	Qué es
Agent	modelo + instrucción + tools opcionales — sin estado propio
Runner	orquesta el ciclo, ejecuta tools, emite el stream de eventos
Session / SessionService	historial + `session.state` compartido, con distintas garantías de persistencia
Pydantic (`output_schema` / `input_schema`)	contratos de datos tipados y validados en runtime
`SequentialAgent` / `ParallelAgent`	encadenar agentes: con dependencia de datos o sin ella
`LiteLlm`	mismo agente, cualquier proveedor de modelo
6 callbacks	observar o interceptar cualquier etapa del ciclo

Los seis módulos configuran una progresión deliberada en la que **cada módulo introduce exactamente una idea adicional** sobre una base ya establecida, sin que ninguna idea posterior invalide a las anteriores.

El pipeline de 5 agentes del final no usa un solo concepto que no haya aparecido en la primera hora. ADK no tiene una curva de complejidad pronunciada — tiene un puñado de piezas simples que se combinan.

Documentación citada

- Get started (Python)
- LLM Agents (`output_schema` , `input_schema` , `output_key`)
- Function tools
- Sequential Agents
- Parallel Agents
- Modelos · LiteLLM
- Runtime (Runner, event loop)
- Sessions · State
- Callbacks
- Referencia API Python

Preguntas frecuentes esperables

- **`Agent` vs `LlmAgent`** — son el mismo tipo; `Agent` es un alias en `google.adk.agents` .
- **¿ `session.state` persiste entre sesiones distintas?** — No, por diseño. Vive dentro de una `Session` identificada por `session_id` .
- **¿Dos agentes de un `ParallelAgent` escribiendo la misma clave?** — Condición de carrera: gana el último. Deben escribir claves disjuntas.
- **¿ `output_schema` con cualquier modelo?** — Con Gemini directo; combinado con `tools` en el mismo agente solo es confiable en Gemini 3.0+.